Satisfiability and SAT Solvers

CS 270 Math Foundations of CS

Jeremy Johnson

Objective

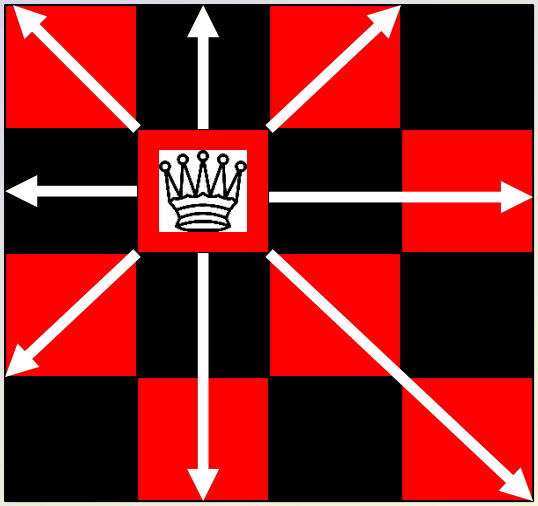
- To show how problems can be reduced to satisfiability and solved using a SAT solver

Outline

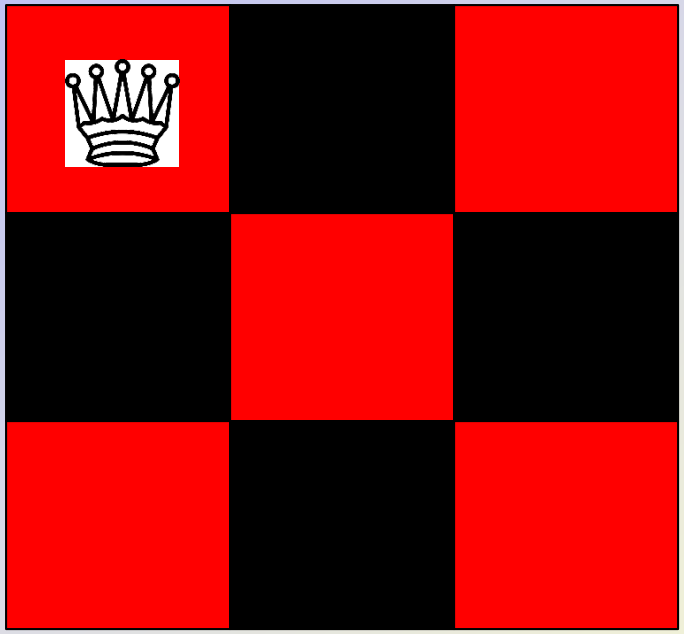
- N Queens Problem
- Backtrack Search
- Satisfiability
- N Queens as at SAT Problem
- SAT Solvers (MiniSAT)
- Solving N Queens using a SAT Solver

N-Queens Problem

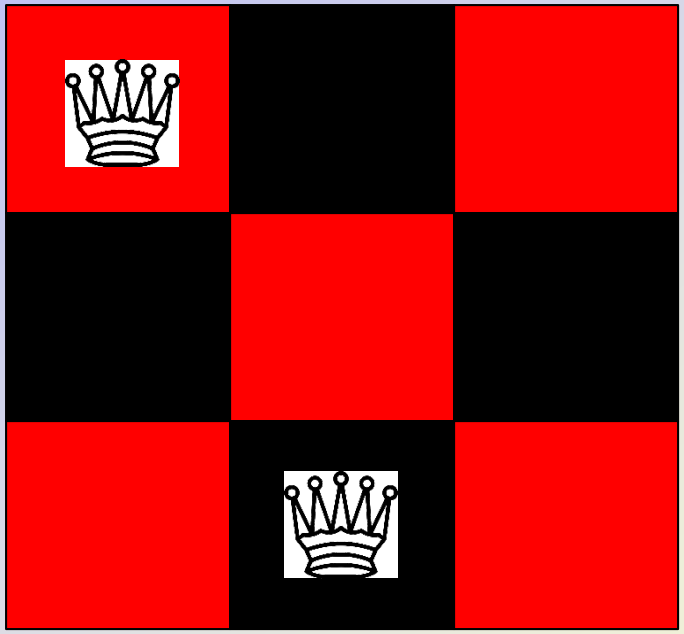
- Given an N x N chess board
  - Find a placement of N queens such that no two queens can take each other



N Queens

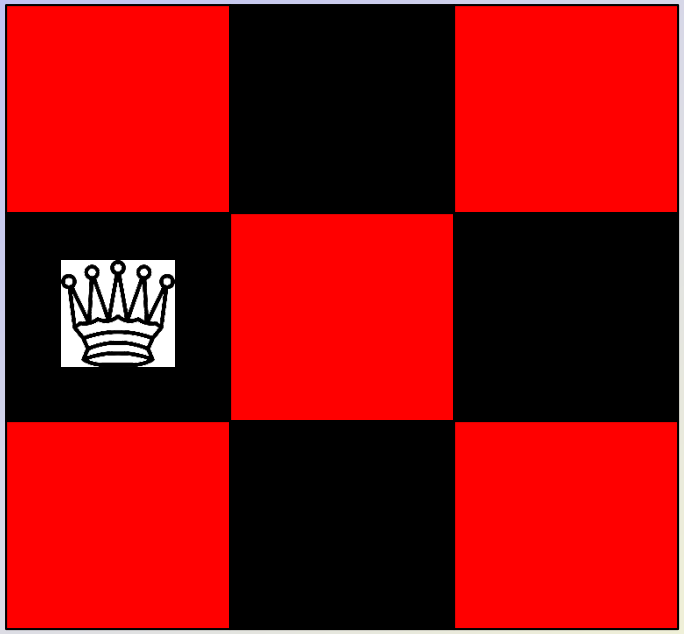


N Queens



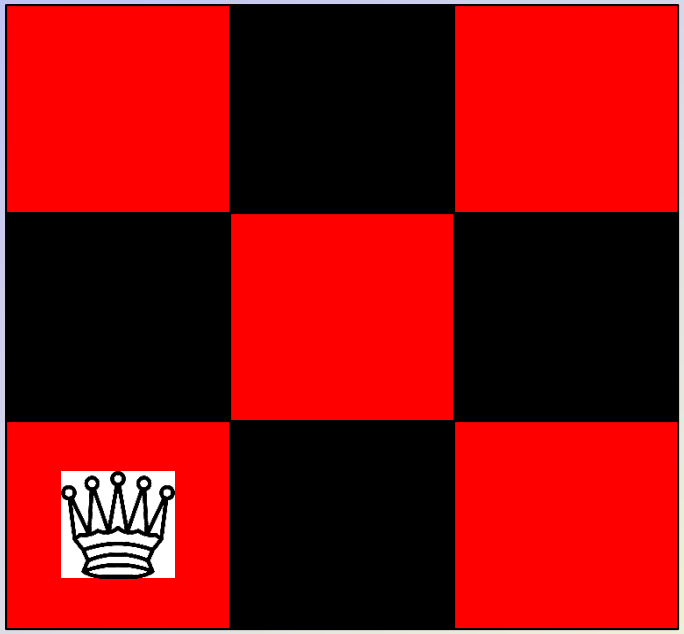
Backtrack

N Queens

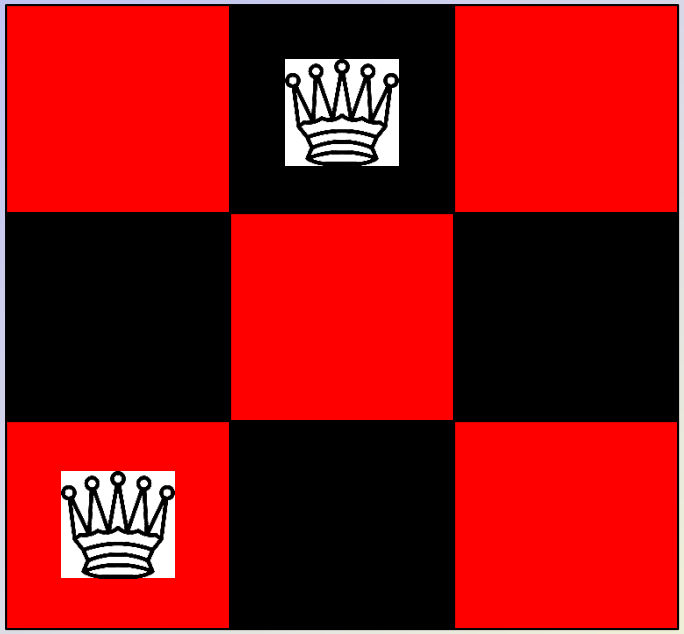


Backtrack

N Queens

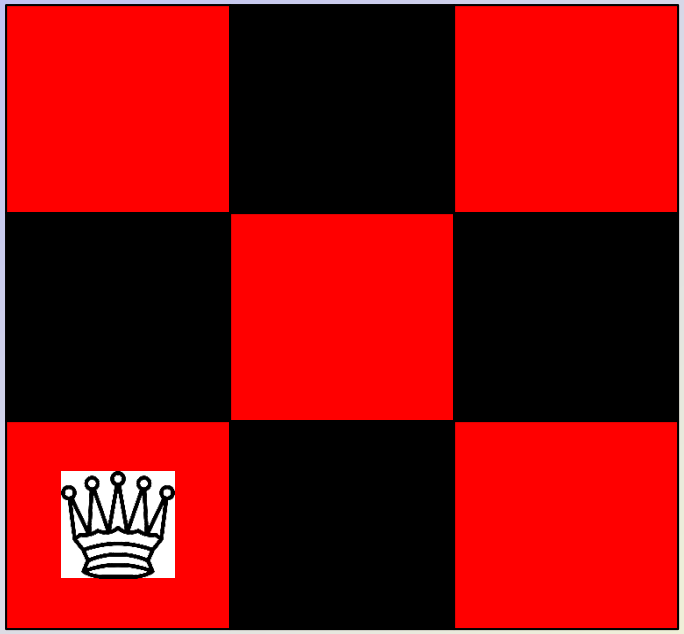


N Queens



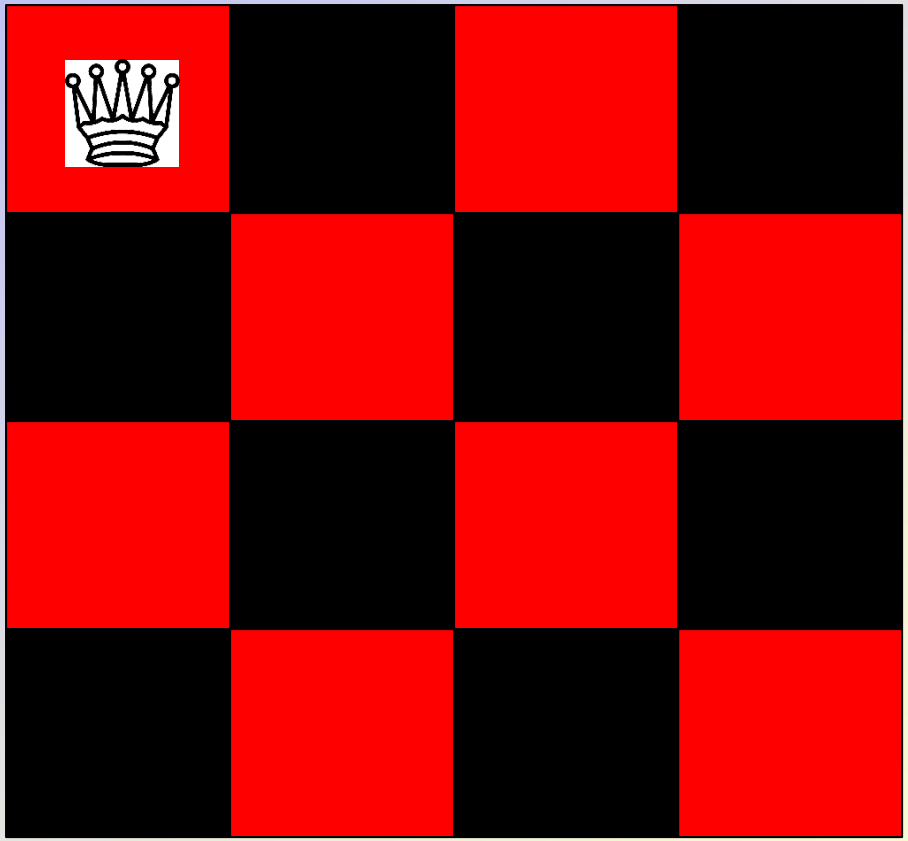
Backtrack

N Queens

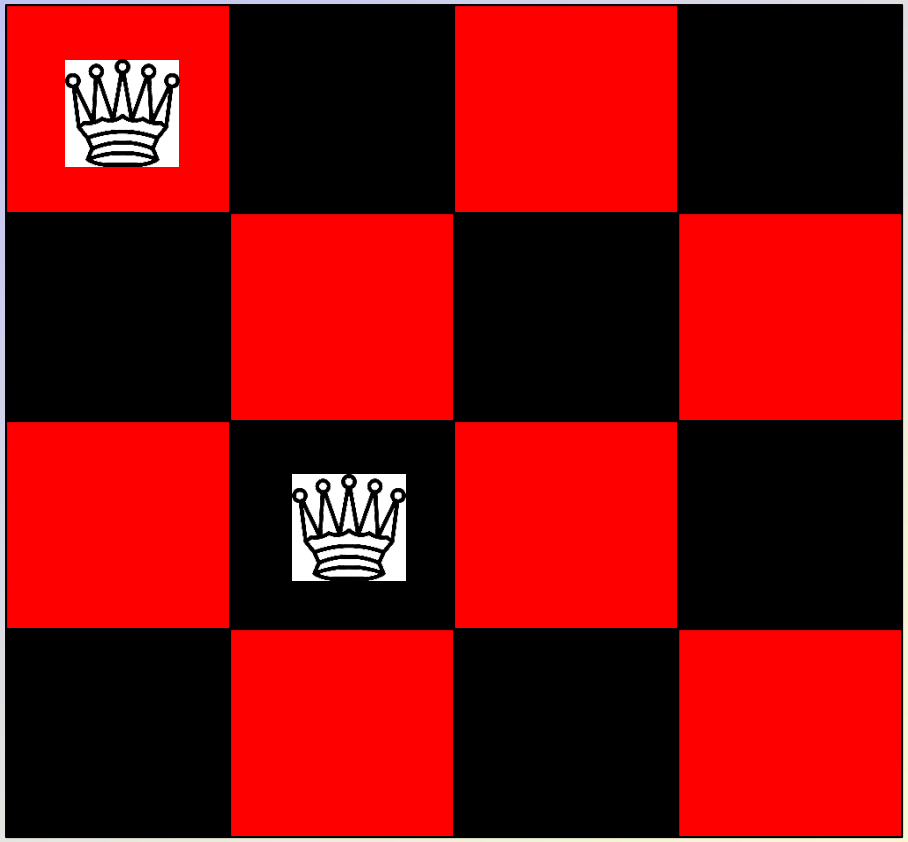


No Solution

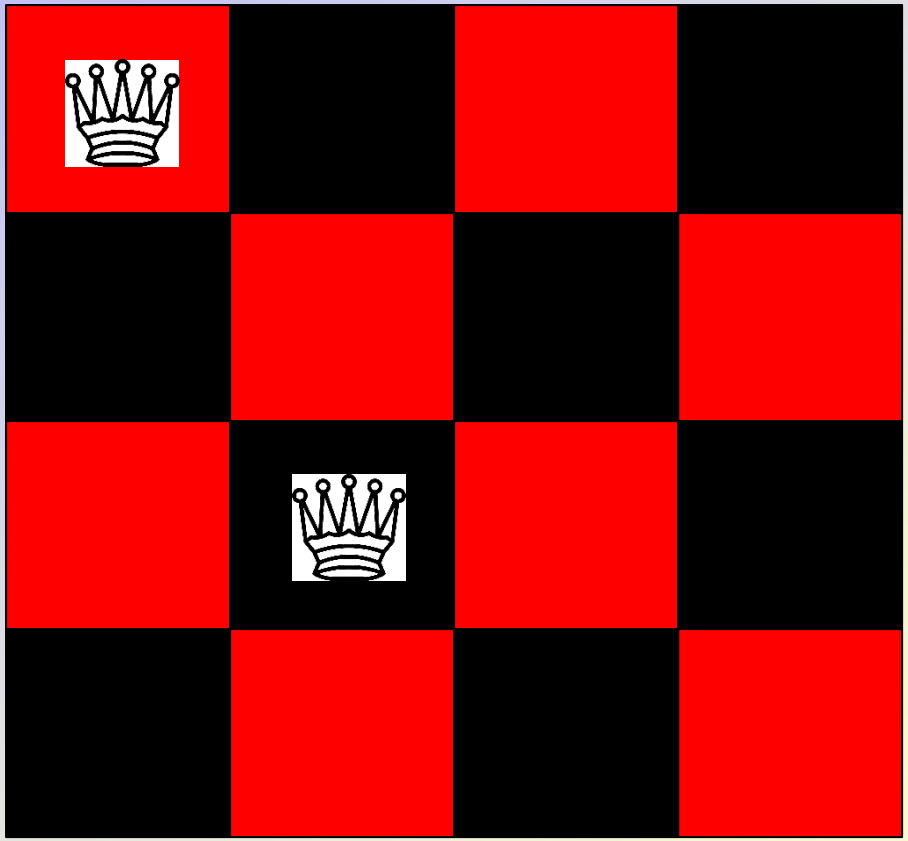
N Queens



N Queens

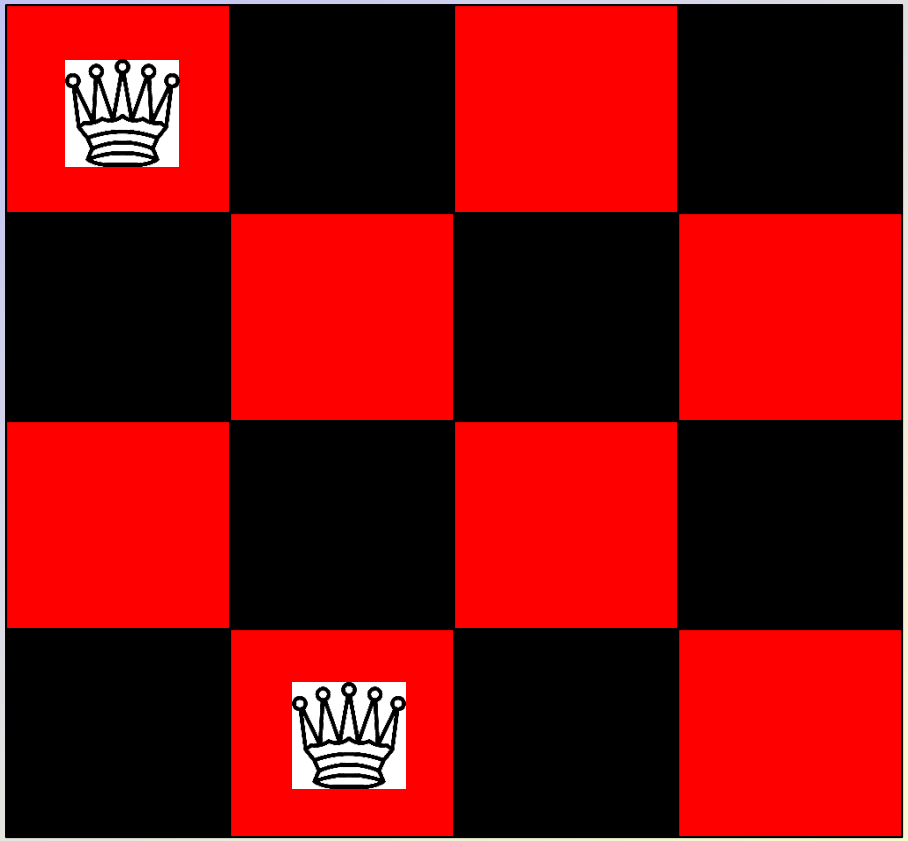


N Queens

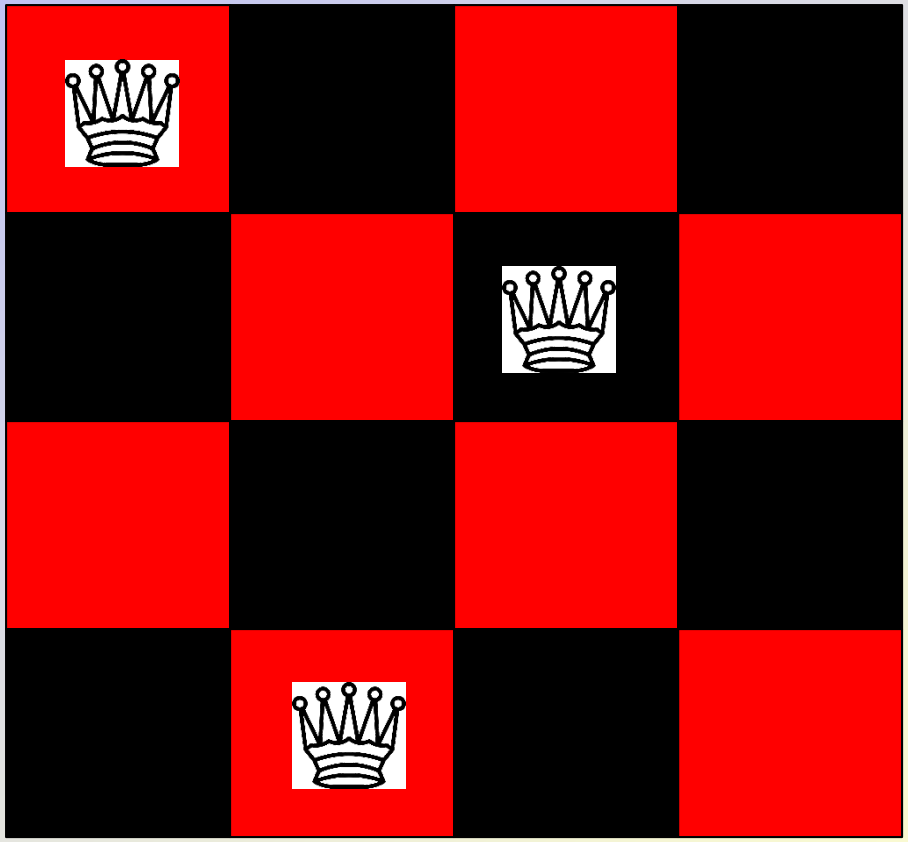


Backtrack

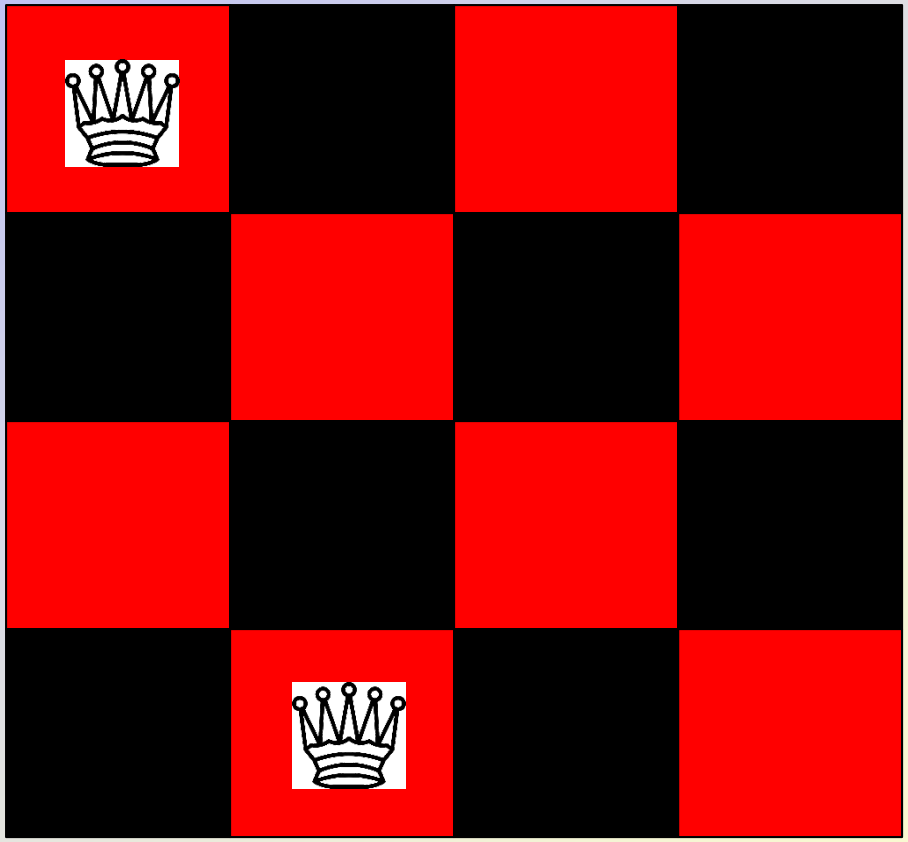
N Queens



N Queens

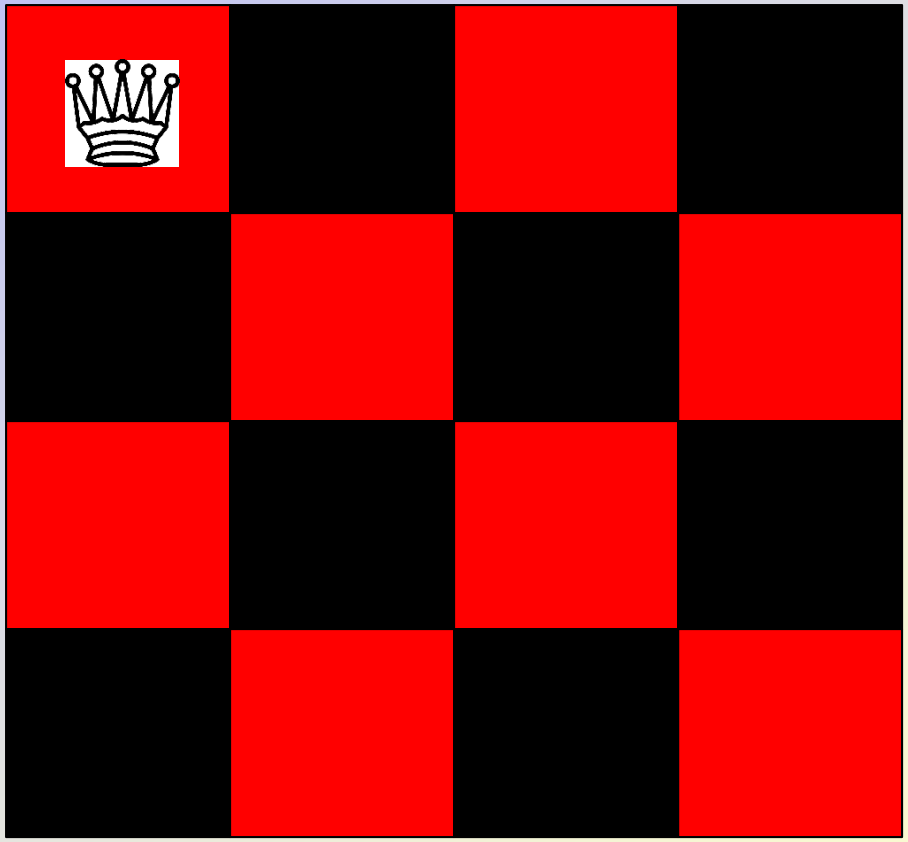


N Queens



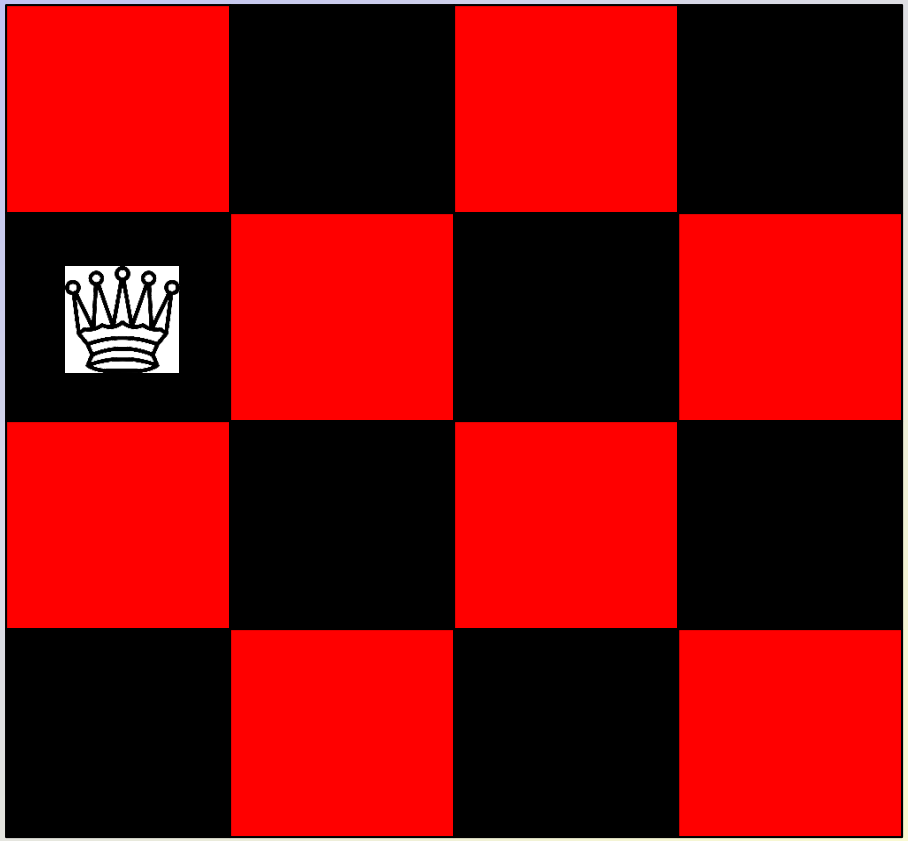
Backtrack

N Queens

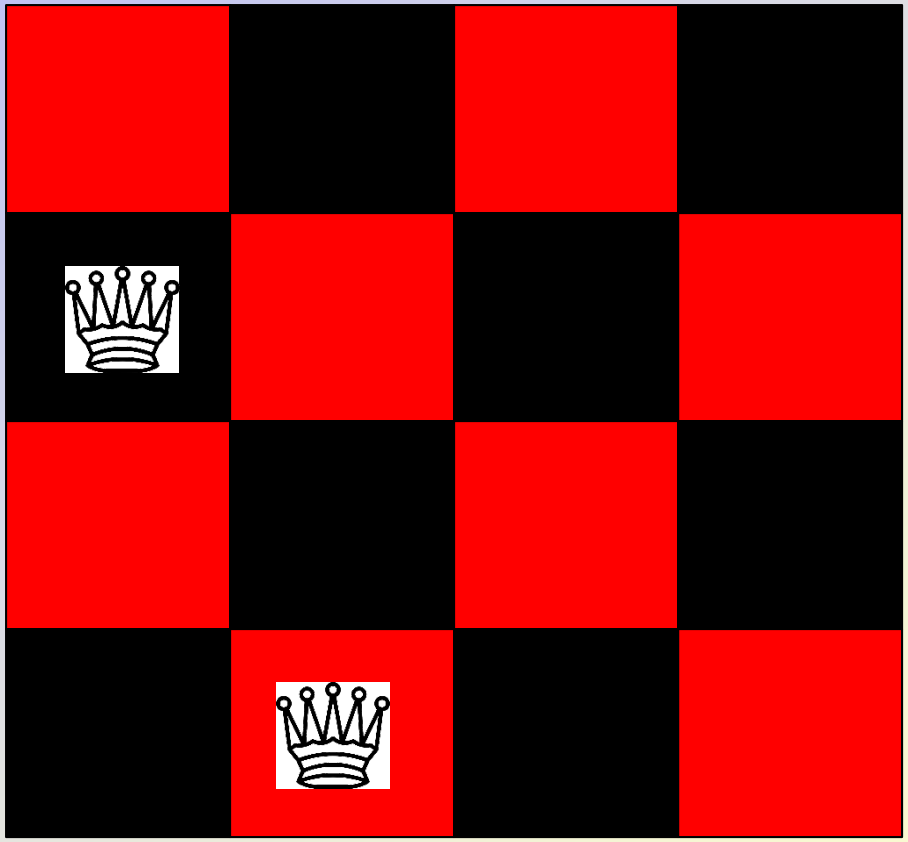


Backtrack

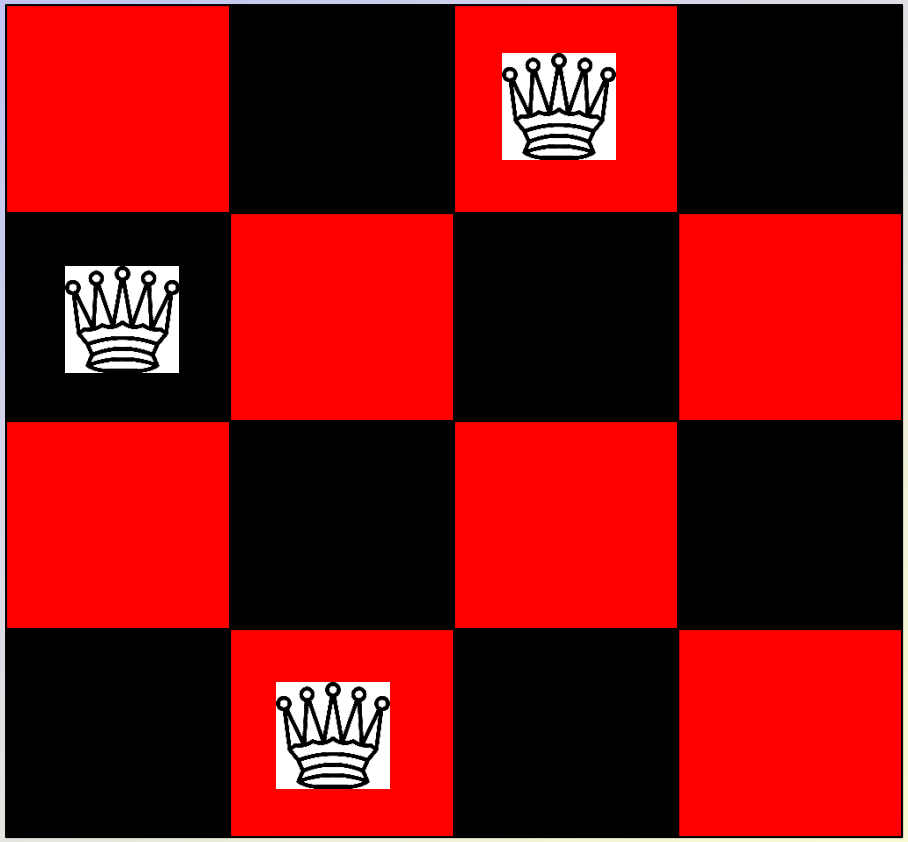
N Queens



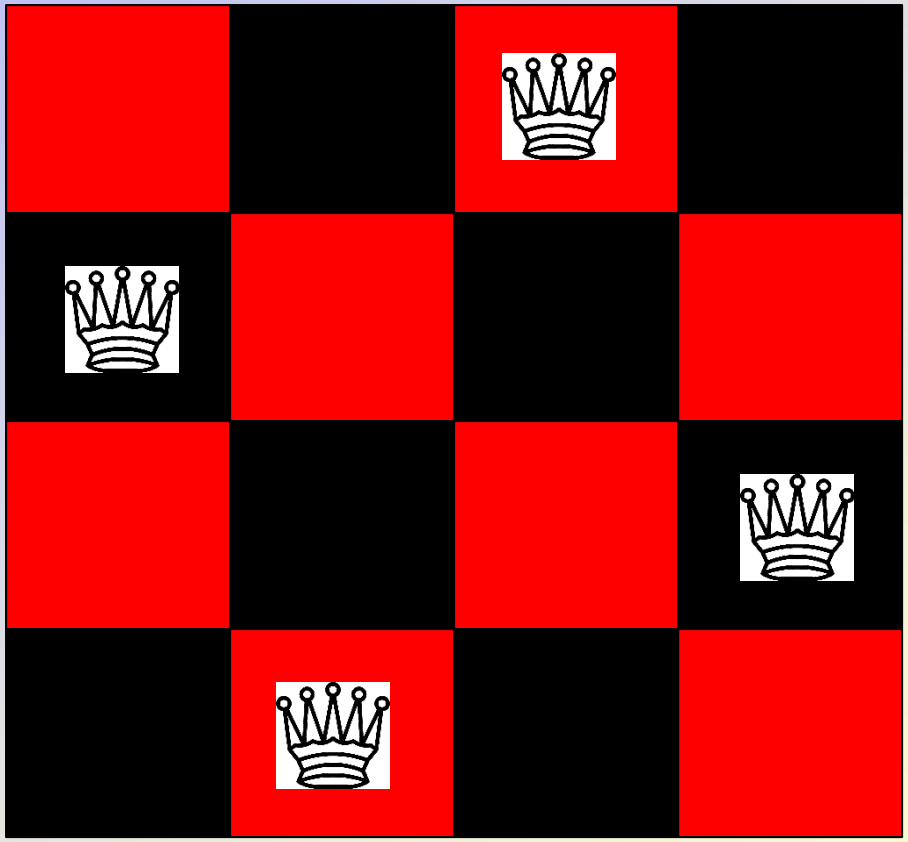
N Queens



N Queens



N Queens



Solution Found

Recursive Solution to N-Queens

- Backtrack search with pruning
- Extend partial solution and prune when this is not possible

```
; Search for solution to the n queens problem.
; Input: n positive integer
; Ouput: a list of pairs (i,j) with 1 <= i,j <= n.
;       equal to the empty list if no solution exists.
;       otherwise a list of queen positions that solves the problem.

(define (nqueens n)
  (extend-nqueens n null 1 1))
```

Recursive Solution to N-Queens

```
; search for a solution that extends the current board configuration
; to include a queen in column j by placing a queen at position (i j).
; retun null if not possible and solution if possible.

(define (extend-nqueens n board i j)
  (cond
    [(> j n) board]                                  ; solution found
    [(> i n) null]                                   ; no extension possible => backtrack

    [(attack? board i j) (extend-nqueens n board (+ i 1) j)]   ; try next placement

    [else (let ((extended_board (extend-nqueens n
                                  (place-queen board i j) 1 (+ j 1))))
            (if (null? extended_board)
                (extend-nqueens n board (+ i 1) j)           ; backtrack
                extended_board))]))                          ; solution found
```

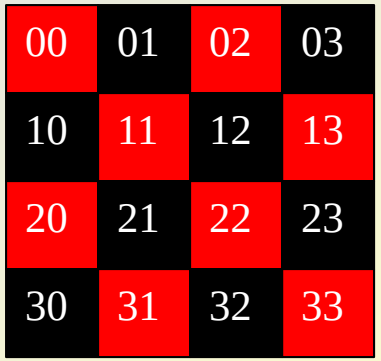
Satisfiability

- A boolean expression is satisfiable if there is an assignment to the variables that makes it true
  - Easy to check hard to find
- Checking to see if a formula f is satisfiable can be done by searching a truth table for a true entry
  - Exponential in the number of variables
  - Does not appear to be a polynomial time algorithm (satisfiability is NP-complete)
  - There are efficient satisfiability checkers that work well on many practical problems

N-Queens as a SAT Problem

- Introduce variables B_{ij} for $0 \leq i,j < N$
  - B_{ij} = T if queen at position (i,j) F otherwise

- Constraints
  - Exactly one queen per row
    - $Row_i = B_{ij}$, j=0…N-1
  - Exactly one queen per column
    - $Column_j = B_{ij}$, i=0…N-1
  - At most one queen on diagonal
    - $Diagonal_{k-} = B_{ij}$, i-j = k = -N+1…,N-1
    - $Diagonal_{k+} = B_{ij}$, i+j = k = 0…,2N-2



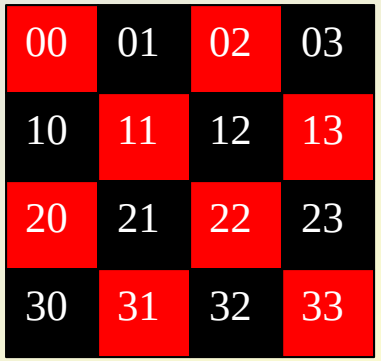
4-Queens SAT input

- Exactly one queen in row i
- $B_{i0} \vee B_{i1} \vee B_{i2} \vee B_{i3}$
- $B_{i0} \rightarrow \neg B_{i1} \wedge \neg B_{i2} \wedge \neg B_{i3}$
- $B_{i1} \rightarrow \neg B_{i2} \wedge \neg B_{i3}$
- $B_{i2} \rightarrow \neg B_{i3}$

00	01	02	03
10	11	12	13
20	21	22	23
30	31	32	33

4-Queens SAT input

- Exactly one queen in column j
- $B_{0j} \vee B_{1j} \vee B_{2j} \vee B_{3j}$
- $B_{0j} \rightarrow \neg B_{1j} \wedge \neg B_{2j} \wedge \neg B_{3j}$
- $B_{1j} \rightarrow \neg B_{2j} \wedge \neg B_{3j}$
- $B_{2j} \rightarrow \neg B_{3j}$

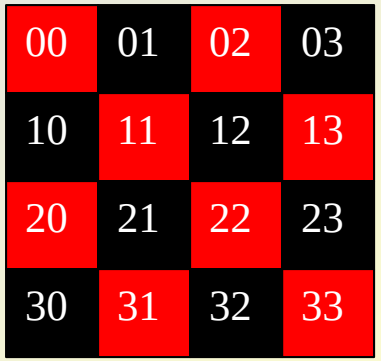


4-Queens SAT input

- At most one queen in diagonal k-
- $B_{20} \rightarrow \neg B_{31}$

  …
- $B_{00} \rightarrow \neg B_{11} \wedge \neg B_{22} \wedge \neg B_{33}$
- $B_{11} \rightarrow \neg B_{22} \wedge \neg B_{33}$
- $B_{22} \rightarrow \neg B_{33}$

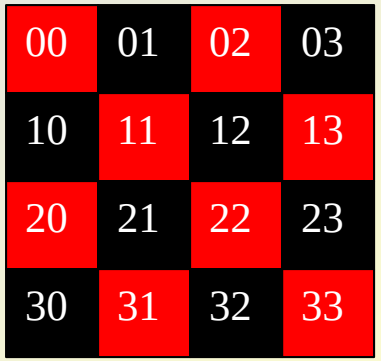
  …
- $B_{02} \rightarrow \neg B_{13}$



4-Queens SAT input

- At most one queen in diagonal k+
- $B_{01} \rightarrow \neg B_{10}$

  …
- $B_{30} \rightarrow \neg B_{21} \wedge \neg B_{12} \wedge \neg B_{03}$
- $B_{21} \rightarrow \neg B_{12} \wedge \neg B_{03}$
- $B_{12} \rightarrow \neg B_{03}$
- …
- $B_{32} \rightarrow \neg B_{23}$



SAT Solvers

- Input expected in CNF
- Using DIMACS format
  - One clause per line delimited by 0
  - Variables encoded by integers, not variable encoded by negating integer
- We will use MiniSAT (minisat.se)

MiniSAT Example

- (x1 | -x5 | x4) & (-x1 | x5 | x3 | x4) & (-x3 | x4).

- DIMACS format
  - (c = comment, “p cnf” = SAT problem in CNF)

```
c SAT problem in CNF with 5 variables and 3 clauses
p cnf 5 3
1 -5 4 0
-1 5 3 4 0
-3 -4 0
```

MiniSAT Example

- (x1 | -x5 | x4) & (-x1 | x5 | x3 | x4) & (-x3 | x4).

```
This is MiniSat 2.0 beta
============================[ Problem Statistics ]=============================
| |
| Number of variables: 5 |
| Number of clauses: 3 |
| Parsing time: 0.00 s |


....


SATISFIABLE
v -1 -2 -3 -4 -5 0
```

Atmost One

- Recall the condition for at most one of the variables $P_1,\ldots,P_t$ to be true

$P_1 \rightarrow (\neg P_2 \wedge \cdots \wedge \neg P_t)$

…

$P_{t-2} \rightarrow (\neg P_{t-1} \wedge \neg P_t)$

$P_{t-1} \rightarrow \neg P_t$

Atmost One

- When converting to CNF we used a generalized version of the distributive law

$$P_1 \rightarrow (\neg P_2 \wedge \cdots \wedge \neg P_t)$$

$$\neg P_1 \vee (\neg P_2 \wedge \cdots \wedge \neg P_t)$$

$$(\neg P_1 \vee \neg P_2) \wedge \cdots \wedge (\neg P_1 \vee \neg P_t)$$

nqueens.py

```
#!/usr/bin/env python
# python script to generate SAT encoding of N-queens problem
#
# Jeremy Johnson and Mark Boady

import sys
#Helper Functions
#cnf formula for exactly one of the variables in list A to be true
def exactly_one(A):
	temp=""
	temp=temp+atleast_one(A)
	temp=temp+atmost_one(A)
	return temp
#cnf formula for atleast one of the variables in list A to be true
def atleast_one(A):
	temp=""
	for x in A:
	  temp = temp +" " +str(x)
	temp=temp+" 0\n"
	return temp
```

nqueens.py

```
#cnf formula for atmost one of the variables in list A to be true
def atmost_one(A):
	temp=""
	for x in A:
	  for y in A[A.index(x)+1:]:
		temp = temp +" -"+str(x)+" -"+str(y)+" 0\n"
	return temp
#function to map position (r,c) 0 <= r,c < N, in an NxN grid to the integer
# position when the grid is stored linearly by rows.
def varmap(r,c,N):
    return r*N+c+1

#Read Input
if len(sys.argv)>1:
	N=int(sys.argv[1])
else:
	N=3
#Check for Sane Input
if N<1:
	print("Error N<1")
	sys.exit(0)
```

nqueens.py

```
#Start Solver
print("c SAT Expression for N="+str(N))
spots = N*N
print("c Board has "+str(spots)+" positions")

#Exactly 1 queen per row
temp=""
for row in range(0,N):
        A=[]
        for column in range(0,N):
          position = varmap(row,column,N)
          A.append(position)
        temp = temp+exactly_one(A)
#Exactly 1 queen per column
for column in range(0,N):
        A=[]
        for row in range(0,N):
          position = varmap(row,column,N)
          A.append(position)
        temp = temp+exactly_one(A)
```

nqueens.py

```
#Atmost 1 queen per negative diagonal from left
for row in range(N-1,-1,-1):
        A=[]
        for x in range(0,N-row):
          A.append(varmap(row+x,x,N))
        temp=temp+atmost_one(A)
#Atmost 1 queen per negative diagonal from top
for column in range(1,N):
        A=[]
        for x in range(0,N-column):
          A.append(varmap(x,column+x,N))
        temp=temp+atmost_one(A)
#Atmost 1 queen per positive diagonal from right
for row in range(N-1,-1,-1):
        A=[]
        for x in range(0,N-row):
          A.append(varmap(row+x,N-1-x,N))
        temp=temp+atmost_one(A)
#Atmost 1 queen per positive diagonal from top
for column in range(N-2,-1,-1):
        A=[]
        for x in range(0,column+1):
          A.append(varmap(x,column-x,N))
        temp=temp+atmost_one(A)
print 'p cnf ' + str(N*N) + ' ' + str(temp.count('\n')) + '\n'
print(temp)
```

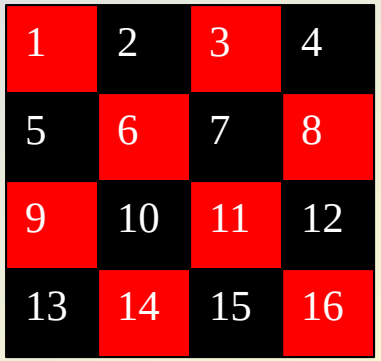
00	01	02	03
10	11	12	13
20	21	22	23
30	31	32	33

4-Queens DIMACS Input (rows)

c SAT Expression for N=4
c Board has 16 positions
p cnf 16 84

1 2 3 4 0
-1 -2 0
-1 -3 0
-1 -4 0
-2 -3 0
-2 -4 0
-3 -4 0
5 6 7 8 0
-5 -6 0
-5 -7 0
-5 -8 0
-6 -7 0
-6 -8 0
-7 -8 0

9 10 11 12 0
-9 -10 0
-9 -11 0
-9 -12 0
-10 -11 0
-10 -12 0
-11 -12 0
13 14 15 16 0
-13 -14 0
-13 -15 0
-13 -16 0
-14 -15 0
-14 -16 0
-15 -16 0

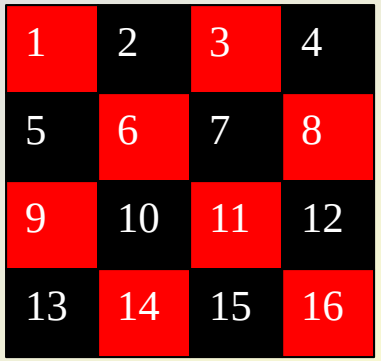


4-Queens DIMACS Input (cols)

```
c SAT Expression for N=4
c Board has 16 positions
p cnf 16 84

 1 5 9 13 0
-1 -5 0
-1 -9 0
-1 -13 0
-5 -9 0
-5 -13 0
-9 -13 0
2 6 10 14 0
-2 -6 0
-2 -10 0
-2 -14 0
-6 -10 0
-6 -14 0
-10 -14 0

3 7 11 15 0
-3 -7 0
-3 -11 0
-3 -15 0
-7 -11 0
-7 -15 0
-11 -15 0
4 8 12 16 0
-4 -8 0
-4 -12 0
-4 -16 0
-8 -12 0
-8 -16 0
-12 -16 0
```

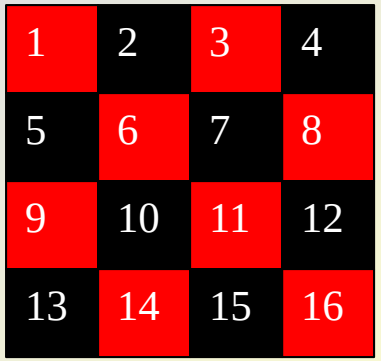


4-Queens DIMACS Input (diag)

c SAT Expression for N=4
c Board has 16 positions
p cnf 16 84

-9 -14 0
-5 -10 0
-5 -15 0
-10 -15 0
-1 -6 0
-1 -11 0
-1 -16 0
-6 -11 0
-6 -16 0
-11 -16 0
-2 -7 0
-2 -12 0
-7 -12 0
-3 -8 0

-12 -15 0
-8 -11 0
-8 -14 0
-11 -14 0
-4 -7 0
-4 -10 0
-4 -13 0
-7 -10 0
-7 -13 0
-10 -13 0
-3 -6 0
-3 -9 0
-6 -9 0
-2 -5 0



4-Queens Output

SAT

-1 -2 3 -4 5 -6 -7 -8 -9 -10 -11 12 -13 14 -15 -16 0

